

计算机系统基础

实验报告

Lab 1 DataLab

姓名：肖飞扬

学 号：25303050026

日期：2026 年 10 月 8 日

一、实验目的

本次 DataLab 实验完成 `bits.c` 中 P1~P19 共 19 个函数，内容包括补码、掩码与移位、溢出判断、位计数和单精度浮点数的位级运算。实验要求在限定运算符和操作数数量的条件下实现指定功能，并同时通过规则检查与正确性测试。最终 19 个函数均通过操作符检查，`btest` 各项 Errors 为 0，总分为 110/110。

通过本实验，主要理解了如何用符号掩码代替分支选择，如何在不使用更宽整数的条件下处理溢出，以及如何依据浮点数的符号、阶码和小数位完成规格化与向最接近偶数舍入。

二、实验环境与编码要求

实验在 WSL 的 Linux 终端中完成，目录为 `/mnt/d/ICS/Lab/Lab1`，对应 Windows 路径 `D:\ICS\Lab\Lab1`。规则检查使用 Python 3 运行 `check_ops.py`，正确性检查使用实验提供的 `btest`。报告中的测试截图分别记录这两项检查结果。

整数题按 32 位二进制补码及算术右移模型解释，超出范围的运算保留低 32 位，移位量限制在 0~31。除 `bitXor` 只能使用 `~` 和 `&` 外，整数题允许 `!`、`~`、`&`、`^`、`|`、`+`、`<<`、`>>`，常量限制为 0~255，不使用分支、循环、比较和类型转换。浮点题允许 `int`、`unsigned` 以及整数控制流，但仍不能直接使用浮点类型或浮点运算。

三、终端检查与测试结果

1 运算符与操作数检查

输出显示各题操作数均在上限内，最后提示 19 个函数全部通过规则检查。

```
mnjmy@NaOH:/mnt/d/ICS/Lab/Lab1$ python3 ./check_ops.py ./bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 5/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 14/24 operators
midpointTowardFirst: 19/32 operators
isBetweenEitherOrder: 25/48 operators
mul5Sat: 17/30 operators
classifyAdd3: 19/52 operators
floatScaleThreeHalves: 35/60 operators
floatRoundEven: 25/65 operators
float_i2f: 22/40 operators
bitCount: 28/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

图 1 check_ops.py 对 bits.c 的规则检查结果

2 btest 正确性测试

运行 ./btest, 19 个函数的 Errors 均为 0，最终为 Total points: 110/110。

```
mnjmy@NaOH:/mnt/d/ICS/Lab/Lab1$ ./btest
Score    Rating  Errors  Function
1         1        0      signMask
2         2        0      bitXor
3         3        0      negativePart
4         4        0      copyByteWithin
4         4        0      logicalShift
4         4        0      swapNibblePairs
4         4        0      secondLowestZeroBit
5         5        0      oddParity
5         5        0      rotateRightBits
5         5        0      roundEvenPow2
5         5        0      midpointTowardFirst
7         7        0      isBetweenEitherOrder
7         7        0      mul5Sat
7         7        0      classifyAdd3
7         7        0      floatScaleThreeHalves
10        10        0      floatRoundEven
10        10        0      float_i2f
10        10        0      bitCount
10        10        0      bitReverse
Total points: 110/110
```

图 2 btest 正确性测试结果

四、各函数实现思路

以下按照 bits.c 的函数顺序说明实现方法。代码框给出关键表达式或主要步骤，解释性注释用于省略重复过程；运算符数量以图 1 的检查结果为准。涉及加法溢出和符号位左移的表达式均按课程规定的 32 位位向量模型理解。

1 signMask 最高位掩码

将常量 1 左移 31 位，得到只有最高位为 1 的位模式 0x80000000。这里构造的是符号位掩码，使用的常量仍在 0~255 的允许范围内；按实验的 32 位位向量模型解释，该结果对应 INT_MIN。

```
return 1 << 31;
```

2 bitXor 仅使用取反与按位与实现异或

$A \oplus B = (A \vee B) \wedge \neg(A \wedge B)$ 而 $A \vee B = \neg(\neg A \wedge \neg B)$ 故 $A \oplus B = \neg(\neg A \wedge \neg B) \wedge \neg(A \wedge B)$

因此最终表达式为 $\sim(\sim x \& \sim y) \& \sim(x \& y)$ ，且实现中只使用 $\sim$ 和 $\&$ 。

```
return ~(~x&~y)&~(x&y);
```

3 negativePart 利用符号掩码提取负数部分

算术右移 $x \gg 31$ 会把符号位扩展到全部 32 位：负数得到全 1，非负数得到全 0。用 $\sim x + 1$ 构造 x 的补码相反数，再与符号掩码相与，即可在 x 为负时返回 $-x$ ，其他情况返回 0。

当 x 为 INT_MIN 时，数学上的相反数无法用 int 表示；按本实验的位向量规则保留低 32 位，结果仍是 0x80000000。

```
return (x>>31)&(~x+1);
```

4 copyByteWithin 提取源字节并替换目标字节

字节编号乘以 8 就是对应的位偏移，因此分别用 $\text{src} \ll 3$ 和 $\text{dst} \ll 3$ 计算偏移。先将源字节移至最低 8 位并与 255 相与，去掉其他位；再构造 $255 \ll \text{targetShift}$ ，把目标字节清零，最后将提取的字节左移后合并。

源字节在修改前已从原始 x 中提取，所以 src 与 dst 相同也能返回原值。例如把 0x11223344 的第 0 字节复制到第 2 字节，结果为 0x11443344。

```
int byte = (x >> sourceShift) & 255;
return (x & ~mask) | (byte << targetShift);
```

5 logicalShift 通过掩码清除符号扩展位

有符号 int 的右移会补符号位，需要在右移后清除多出来的高位 1。表达式 $\sim(((1 \ll 31) \gg n) \ll 1)$ 构造低 32-n 位为 1 的掩码，将它与 $x \gg n$ 相与即可得到逻辑右移结果。

当 n 为 0 时，掩码为全 1，结果保持 x ；当 n 为 31 时，掩码只保留最低位。整个过程没有使用 32 位的移位置。

```
int mask = ~(((1 << 31) >> n) << 1);
return (x >> n) & mask;
```

6 swapNibblePairs 交换每个字节中的两个半字节

先用 15 及其移位组合构造 0x0F0F0F0F。 $x \& \text{mask}$ 提取各字节的低 4 位，左移 4 位后成为高半字节； $(x \gg 4) \& \text{mask}$ 提取各字节原来的高 4 位。将两部分按位或，就同时完成四个字节内部的交换。

右移产生的符号扩展会被掩码清除，且左移前只保留低半字节，因此各字节之间不会相互混入。例如 0xAB 变为 0xBA。

```
return ((x & mask) << 4) | ((x >> 4) & mask);
```

7 secondLowestZeroBit 先填补最低零位再定位下一个零位

对 x 加 1 会把末尾连续的 1 清零，并把其后的第一个 0 变为 1。 $x | (x + 1)$ 保留原来的 1，同时填上最低的 0。设结果为 filled ，则 $\sim \text{filled} \& (\text{filled} + 1)$ 恰好提取 filled 中最低的 0，也就是原始 x 中第二低的 0。

如果原值只有一个 0，填补后为全 1；如果原值本来就是全 1，也不会找到目标位。两种情况都会返回 0。例如 0xFFFFF0 的两个最低零位在第 0、2 位，结果为 0x4。

```
int filled = x | (x + 1);
return ~filled & (filled + 1);
```

8 oddParity 通过异或折叠求奇校验位

依次将 x 与右移 16、8、4、2、1 位的结果异或，把全部 32 位的异或值折叠到最低位。最低位为 1 表示原数中 1 的个数为奇数，为 0 表示个数为偶数。题目要求返回补上的奇校验位，因此再与 1 相与并异或 1。

例如 5 有两个 1，需补上 1 才能形成奇数个 1，所以返回 1；7 有三个 1，返回 0。算术右移的高位补位不影响最终最低位对原始 32 位的折叠结果。

```
x = x ^ (x >> 16); /* 再依次折叠 8、4、2、1 位 */
return (x & 1) ^ 1;
```

9 rotateRightBits 拼接逻辑右移部分与回绕部分

循环移位以 32 为周期，先执行 $n \& 31$ ，将任意非负 n 归约到 0~31。低位向高位回绕所需的左移量为 $(32-n) \bmod 32$ ，用 $(\sim n + 1) \& 31$ 构造；右移部分复用逻辑右移的掩码，最后按位或拼接两部分。

n 为 0 或 32 的倍数时，两部分都是原值，结果保持不变，也避免了左移 32 位。例如 0x12345678 循环右移 8 位得到 0x78123456。

```
n = n & 31;
leftShift = (~n + 1) & 31;
return ((x >> n) & mask) | (x << leftShift);
```

10 roundEvenPow2 用偏置实现最接近偶数舍入

把 x 看成商 $q=x>>n$ 与低 n 位余数的组合。通常可以加上半程后清零低 n 位，但恰好位于中点时，需要根据 q 的奇偶决定是否进位。实现采用偏置 $2^{(n-1)}-1$ ，再额外加 q 的最低位。

余数小于半程时不会进位，大于半程时一定进位；余数等于半程时， q 为偶数则保留，为奇数则进位到下一个偶数商。最后与 $\sim((1 << n) + \sim 0)$ 相与，清除低 n 位。例如 $n=2$ 时，10 舍入到 8，14 舍入到 16。

```
int bias = (1 << (n + ~0)) + ~0;
return (x + bias + (quotient & 1)) & mask;
```

11 midpointTowardFirst 避免加法溢出并向第一个参数取整

直接计算 $(x+y)/2$ 可能在相加时溢出。利用恒等式 $x+y=2\times(x\&y)+(x\^y)$ ，先计算 $(x\&y)+((x\^y)>>1)$ ，得到数学中点的向下取整值，避免直接求和的溢出。

当 x 与 y 奇偶不同，中点位于两个整数之间。此时若 x 较大，需要把向下取整结果加 1，才能更接近第一个参数。比较时先看两个符号：异号由符号决定，同号再看 $x-y$ 的符号，防止跨符号减法溢出造成误判。例如 (4,7) 返回 5，交换参数后返回 6。

```
int average = (x & y) + (different >> 1);
return average + (firstIsGreater & different & 1);
```

12 isBetweenEitherOrder 用两次安全比较判断无序区间

分别构造 $x<a$ 与 $x<b$ 的全位掩码。比较表达式在符号不同的时候选择 x 的符号位，在符号相同的时候选择 $x-a$ 或 $x-b$ 的符号位，这样既能比较极值，又不会因异号相减溢出而误判。

若 x 严格位于两个端点之间，两次比较的结果一定不同，因此 $\text{lessA} \wedge \text{lessB}$ 非零；再用 $!\text{differentA}$ 和 $!\text{differentB}$ 补上等于任一端点的情况。最终结果为 0 或 1，且交换 a 、 b 不影响结果，端点相同也能正确判断。

```
int lessA = ((differentA & x)
             | (~differentA & (x + (~a + 1)))) >> 31;
return !(lessA ^ lessB) | !differentA | !differentB;
```

13 mul5Sat 分阶段判断乘五溢出并饱和

将乘五拆成 $x+x$ 、 $2x+2x$ 、 $4x+x$ 。每一步都在将同方向的数相加；如果某一步的符号发生变化，就说明超出了有符号整数的表示范围。将相邻结果的异或按位或，再算术右移 31 位，可得到任一步溢出的全位掩码。

饱和值由原始 x 的符号决定： $(1 \ll 31) \wedge \sim(x \gg 31)$ 在非负输入时为 `INT_MAX`，在负输入时为 `INT_MIN`。最后用 `overflow` 掩码选择饱和值或正常乘积。分阶段检查可以捕捉中间发生溢出、最终符号又变回原符号的情况。

```
int overflow = ((x ^ twice) | (twice ^ fourTimes)
               | (fourTimes ^ fiveTimes)) >> 31;
return (overflow & limit) | (~overflow & fiveTimes);
```

14 classifyAdd3 累加两次有符号溢出的修正量

先计算 `firstSum=x+y`，再计算 `finalSum=firstSum+z`。对于一次加法， $\sim(a \wedge b) \wedge (a \wedge \text{sum})$ 的符号位为 1，表示两个同号数相加后结果异号，即发生溢出。正溢出记为 +1，负溢出记为 -1，未溢出记为 0。

每个修正量表示数学结果相对于低 32 位有符号结果多出或少了一个 2^{32} 。两次修正相加，便能分类真实的三数之和。不能把两次溢出简单做按位或，因为中间的正溢出可能被后续负溢出抵消。例如 `INT_MAX+INT_MAX+INT_MIN` 的真实结果为 2147483646，应返回 0。

```
int firstOverflow = (~(x ^ y) & (x ^ firstSum)) >> 31;
int firstCorrection = firstOverflow & ((x >> 31) | 1);
return firstCorrection + finalCorrection;
```

15 floatScaleThreeHalves 对浮点有效数乘三并统一舍入

先拆出符号位、8 位阶码和 23 位小数位。阶码为 255 时直接返回原位模式，保留 NaN 与无穷大。非规格化数没有隐藏的最高位，直接对 `fraction×3` 除以 2；若丢弃位为 1 且保留结果为奇数，则加 1，实现中点向偶数舍入。结果越过非规格化边界时，打包的位模式会自然进入规格化数范围，零的符号也被保留。

规格化数先补回隐藏的 1，形成 24 位有效数再乘 3。默认右移 1 位；若乘积达到 `0x2000000`，则右移 2 位并将阶码加 1。通过余数与半程比较完成一次舍入：大于半程，或等于半程且保留值为奇数时进位。使用加法打包有效数与阶码，使舍入进位能传播到阶码；阶码溢出则返回同符号无穷大。

```
product = (fraction | 0x800000u) * 3;
/* 根据 product 选择 shift=1 或 2, 并执行偶数舍入 */
return sign | (((exponent - 1) << 23) + significand);
```

16 floatRoundEven 按阶码分区处理整数舍入

单精度阶码偏置为 127。存储阶码不小于 150 时，实际指数至少为 23，所有有限值已没有小数位；这个分支也包括 NaN 和无穷大，均原样返回。阶码小于 126 时，绝对值小于 0.5，返回保留符号的零；阶码为 126 时，恰好 ± 0.5 舍入到 ± 0 ，其他值舍入到 ± 1 。

对剩余情况，用 `shift=150-exponent` 确定需要清除的低位数。先清零这些位得到整数候选，再判断被清除的余数是否超过半程，或是否恰好等于半程且候选为奇数；满足时加

$1 \ll \text{shift}$ 。直接在幅值编码上完成进位，符号位保持不变。例如 1.5 舍入到 2.0，2.5 舍入到 2.0，-0.5 得到 -0。

```
shift = 150 - exponent;
result = uf & ~mask;
if ((remainder > halfway)
    || ((remainder == halfway) && ((result >> shift) & 1)))
    result = result + (1u << shift);
```

17 float_i2f 整数幅值规格化后构造浮点编码

零单独返回 0。其他输入先记录符号，并在 unsigned 幅值上通过 $\sim \text{magnitude} + 1$ 取得负数的绝对值，因此 INT_MIN 的幅值 0x80000000 也可以表示。将最高的 1 逐步左移到第 31 位，阶码从 158 (127+31) 开始，每左移一次减 1。

隐藏位位于最高位后，将其去掉并保留后续 23 位作为 fraction，低 8 位作为舍入余数。余数大于 128 时进位，等于 128 时仅在 fraction 为奇数时进位；如果小数位进位溢出，再把小数位清零、阶码加 1。最后合并符号、阶码和小数位，完成向最接近偶数舍入的 int 到 float 位级转换。

```
fraction = (magnitude >> 8) & 0x7fffffu;
remainder = magnitude & 255;
return sign | (exponent << 23) | fraction;
```

18 bitCount 并行分组累加各位的计数

用 85、51、15 及其移位组合构造 0x55555555、0x33333333、0x0F0F0F0F。先将相邻两位的 1 的数量相加，使每个 2 位组保存计数；再合并为 4 位组，随后合并为每个字节的计数。

继续通过右移 8、16 位相加，把四个字节的计数累积到低位。32 位整数最多有 32 个 1，用 $x \& 63$ 提取低 6 位即可得到结果。掩码会清除算术右移产生的符号扩展，所以负数同样按完整 32 位统计，例如 -1 返回 32。

```
x = (x & mask1) + ((x >> 1) & mask1);
x = (x & mask2) + ((x >> 2) & mask2);
x = (x + (x >> 4)) & mask4;
return x & 63;
```

19 bitReverse 按一二四八十六位逐级交换

将整段反转分解为五层交换：相邻 1 位交换、相邻 2 位组交换、半字节交换、字节交换，最后交换高低 16 位。每一层都用掩码提取一侧并移至另一侧，再按位或合并；右移的一侧经过掩码清除符号扩展。为满足 34 次操作的上限，先构造 $\text{mask8} = 0x00FF00FF$ ，再依次用异或与左移导出 $\text{mask4} = 0x0F0F0F0F$ 、 $\text{mask2} = 0x33333333$ 、 $\text{mask1} = 0x55555555$ ，避免分别展开四个掩码。五层交换后，第 i 位移动到第 $31-i$ 位。例如 0x80000004 反转后为 0x20000001。

```
int mask4 = mask8 ^ (mask8 << 4);
int mask2 = mask4 ^ (mask4 << 2);
int mask1 = mask2 ^ (mask2 << 1);
return ((x >> 16) & mask16) | (x << 16);
```

五、实验结果与分析

所有函数均在操作数限制内通过检查，且 `btest` 未报告错误。`bitReverse` 使用 34/34 次操作，达到上限，因此掩码的复用是实现中的关键；其余函数均留有不同程度的余量。下表汇总图 1 与图 2 中的结果。

编号	函数	操作数 已用/上限	得分/满分	Errors
P1	<code>signMask</code>	1/2	1/1	0
P2	<code>bitXor</code>	7/8	2/2	0
P3	<code>negativePart</code>	4/6	3/3	0
P4	<code>copyByteWithin</code>	9/12	4/4	0
P5	<code>logicalShift</code>	6/20	4/4	0
P6	<code>swapNibblePairs</code>	9/18	4/4	0
P7	<code>secondLowestZeroBit</code>	5/8	4/4	0
P8	<code>oddParity</code>	12/56	5/5	0
P9	<code>rotateRightBits</code>	12/16	5/5	0
P10	<code>roundEvenPow2</code>	14/24	5/5	0
P11	<code>midpointTowardFirst</code>	19/32	5/5	0
P12	<code>isBetweenEitherOrder</code>	25/48	7/7	0
P13	<code>mul5Sat</code>	17/30	7/7	0
P14	<code>classifyAdd3</code>	19/52	7/7	0
P15	<code>floatScaleThreeHalves</code>	35/60	7/7	0
P16	<code>floatRoundEven</code>	25/65	10/10	0
P17	<code>float_i2f</code>	22/40	10/10	0
P18	<code>bitCount</code>	28/40	10/10	0
P19	<code>bitReverse</code>	34/34	10/10	0

表 1 各函数操作数与正确性测试汇总

规则检查验证代码写法与操作数限制，`btest` 检查计算结果，两项均通过才满足实验要求。110/110 表示本次 `btest` 测试通过，不等同于对全部可能输入的形式化证明。

六、实验总结

本次实验把整数和浮点运算还原为对 32 位编码的处理。符号扩展可以产生选择掩码，分组掩码可以同时处理多个位域；对溢出需要分析符号变化和进位所代表的数学意义，对舍入则需要保留余数并判断中点的奇偶。